

Segmentación Óptima de Contenido mediante Programación Dinámica y un LLM Local

Fabián Almeida

Julio de 2026

Resumen

Se presenta un sistema de segmentación óptima de contenido textual: dada una secuencia de oraciones, el sistema la divide en segmentos contiguos que maximizan la coherencia semántica interna. La coherencia de cada segmento candidato se evalúa con un modelo de lenguaje local (Qwen 2.5-3B-Instruct cuantizado), y la partición óptima se obtiene con un algoritmo de programación dinámica exacto. Sobre un dataset de 6 documentos anotados (66 oraciones, 17 segmentos temáticos de referencia), la mejor configuración recupera exactamente todas las fronteras de referencia ($P_k = 0$, WindowDiff = 0, $F_1 = 1$), superando ampliamente a un baseline de segmentación uniforme.

Índice

1. Descripción del problema	3
2. Modelado formal	3
2.1. Función objetivo	3
2.2. Restricciones	4
3. Dataset	4
3.1. Descripción	4
3.2. Proceso de generación	4
3.3. Por qué cumple las condiciones del problema	4
4. Diseño del algoritmo	5
4.1. Subestructura óptima	5
4.2. Pseudocódigo	5
4.3. Complejidad	6
5. Rol del LLM en el sistema	6
5.1. Configuración	6
5.2. Puntuación por probabilidad de respuesta binaria	6
5.3. Iteración de diseño: por qué no pedir un número directamente	6
6. Metodología experimental	7
6.1. Métricas	7
6.2. Configuraciones comparadas	7
6.3. Protocolo y entorno	7
7. Resultados y análisis	8
7.1. Comparación entre configuraciones	8
7.2. Resultados por documento	8
7.3. Análisis	8

8. Limitaciones y posibles mejoras	9
8.1. Limitaciones	9
8.2. Posibles mejoras	10

1. Descripción del problema

La *segmentación de contenido* consiste en dividir una secuencia de elementos textuales en bloques contiguos temáticamente homogéneos. Es un problema clásico de procesamiento de lenguaje natural con aplicaciones directas: navegación dentro de documentos largos, generación de resúmenes por secciones, indexación para sistemas de recuperación (*chunking* para RAG), subtitulado por temas y análisis de discurso.

Formulado informalmente: dado un texto compuesto por n oraciones que tratan varios temas en sucesión, se desea encontrar los puntos de corte que separan un tema del siguiente, de modo que cada segmento resultante sea internamente coherente (todas sus oraciones hablan de lo mismo) y las oraciones de segmentos distintos tratan temas distintos.

El problema tiene dos dificultades principales:

1. **Medir coherencia semántica.** Decidir si un conjunto de oraciones “habla de lo mismo” requiere comprensión semántica profunda, no solo solapamiento léxico. Aquí es donde interviene el LLM.
2. **Optimizar sobre un espacio exponencial.** Un texto de n oraciones admite 2^{n-1} segmentaciones contiguas posibles. Se necesita un algoritmo que encuentre la óptima sin enumerarlas todas; la estructura del problema (subestructura óptima sobre prefijos) permite resolverlo con programación dinámica en tiempo polinomial.

2. Modelado formal

Sea $S = (s_1, s_2, \dots, s_n)$ la secuencia de oraciones del documento. Una *segmentación* es una partición de S en m bloques contiguos

$$\sigma = (S_{[b_0, b_1)}, S_{[b_1, b_2)}, \dots, S_{[b_{m-1}, b_m)}), \quad 0 = b_0 < b_1 < \dots < b_m = n,$$

donde $S_{[i, j)} = (s_{i+1}, \dots, s_j)$ denota el segmento formado por las oraciones $i+1$ a j .

Sea $\text{coh} : \text{segmento} \rightarrow [0, 1]$ una función de *coherencia semántica interna*, implementada mediante el LLM (sección 5). El valor 1 indica que todas las oraciones del segmento tratan un mismo tema; el valor 0, que mezclan temas sin relación.

2.1. Función objetivo

La calidad de una segmentación σ se define como la coherencia media ponderada por la longitud de cada segmento, con una penalización a los segmentos de una sola oración:

$$J(\sigma) = \sum_{k=1}^m \left[|S_k| \cdot \text{coh}(S_k) - \lambda \cdot \mathbb{I}[|S_k| = 1] \right],$$

con $\lambda = 0,05$. El problema de optimización es

$$\sigma^* = \arg \max_{\sigma \in \Sigma_{L_{\min}, L_{\max}}} J(\sigma),$$

donde $\Sigma_{L_{\min}, L_{\max}}$ es el conjunto de segmentaciones cuyos segmentos tienen entre $L_{\min}$ y $L_{\max}$ oraciones.

Dos decisiones de diseño merecen justificación:

- **Ponderación por longitud.** Si el objetivo fuera la suma simple $\sum_k \text{coh}(S_k)$, fragmentar el texto en n segmentos de una oración (cada uno trivialmente coherente) daría puntuación n , superior a la de cualquier segmentación razonable: el óptimo degeneraría en la partición

trivial. Con la ponderación $|S_k| \cdot \text{coh}(S_k)$, el total es comparable entre segmentaciones ($J \leq n$ siempre), y agrupar 4 oraciones coherentes vale exactamente lo mismo que separarlas ($4 \times 1,0$ en ambos casos), de modo que la coherencia — y no el número de cortes — decide el óptimo.

- **Penalización λ a singletons.** Una oración sola es trivialmente coherente ($\text{coh} = 1$ por definición, sin consultar al LLM). La penalización λ rompe el empate anterior a favor de los segmentos grandes: solo conviene aislar una oración si unirla a cualquier vecino reduce la coherencia del bloque en más de λ .

2.2. Restricciones

$L_{\text{máx}}$ (por defecto 6) acota el tamaño de segmento y, con ello, el número de llamadas al LLM (sección 4); $L_{\text{mín}}$ (por defecto 1) permite forzar segmentos de un tamaño mínimo.

3. Dataset

3.1. Descripción

Se generó manualmente un dataset de 6 documentos en español (`dataset.py`), cada uno con su segmentación de referencia anotada:

Id	Temas	Oraciones	Segmentos ref.	Tamaños
D01	cambio climático, IA, nutrición	12	3	4+4+4
D02	astronomía, economía, deporte	12	3	4+4+4
D03	historia, biología celular	8	2	4+4
D04	música, arquitectura, psicología	12	3	4+4+4
D05	energía renovable, derecho, gastronomía	12	3	4+4+4
D06	cine, océanos, ajedrez	10	3	3+5+2
Total		66	17	

Cuadro 1: Composición del dataset.

3.2. Proceso de generación

Cada documento se construyó concatenando bloques de oraciones escritos expresamente para el dataset, donde cada bloque desarrolla un tema claramente diferenciado (definición, datos cuantitativos, ejemplos y consecuencias del tema). La segmentación de referencia es, por construcción, la frontera entre bloques, lo que elimina la ambigüedad de anotación que suele afectar a los corpus de segmentación.

Los documentos D01–D05 tienen segmentos de tamaño uniforme (4 oraciones). El documento D06 se añadió deliberadamente con segmentos de tamaño *no uniforme* (3+5+2): sin él, un baseline que divide en bloques iguales conociendo el número verdadero de segmentos coincidiría con la referencia por pura construcción del dataset, y la comparación experimental no discriminaría entre métodos (sección 7).

3.3. Por qué cumple las condiciones del problema

- **Segmentos contiguos y sin solapamiento:** cada documento es una secuencia lineal de oraciones y la referencia es una partición contigua, exactamente el objeto que el sistema debe producir.

- **Coherencia interna alta y coherencia cruzada baja:** los temas de cada documento son deliberadamente distantes entre sí (p. ej. astronomía / economía / fútbol), de modo que existe una señal semántica clara que el evaluador LLM debe detectar; al mismo tiempo, dentro de cada bloque todas las oraciones desarrollan el mismo tema.
- **Sin marcas superficiales:** los documentos se evalúan como texto plano corrido, sin saltos de párrafo ni conectores que delaten las fronteras; la única pista disponible es semántica, lo que obliga a usar el LLM y evita que una heurística léxica trivial resuelva el problema.
- **Variedad de tamaños:** hay documentos de 2 y 3 segmentos, de 8 a 12 oraciones, y con segmentos de 2 a 5 oraciones (D06), lo que permite comprobar que el sistema no asume ni un número fijo de segmentos ni un tamaño fijo de segmento.
- **Verdad de referencia objetiva:** al ser generado por bloques, la anotación es unívoca y las métricas (P_k , WindowDiff, F_1) se calculan sin ruido de anotación.

Como limitación, es un dataset pequeño y con fronteras nítidas; la sección 8 discute su extensión a textos con transiciones graduales.

4. Diseño del algoritmo

4.1. Subestructura óptima

El problema tiene subestructura óptima sobre prefijos: si σ^* es una segmentación óptima de las primeras j oraciones y su último segmento es $S_{[i,j]}$, entonces el resto de σ^* es una segmentación óptima de las primeras i oraciones. Esto justifica la recurrencia

$$dp[j] = \max_{i: L_{\min} \leq j-i \leq L_{\max}} \{dp[i] + g(i, j)\}, \quad dp[0] = 0,$$

donde la ganancia del segmento candidato $S_{[i,j]}$ es

$$g(i, j) = \begin{cases} 1 - \lambda & \text{si } j - i = 1, \\ (j - i) \cdot \text{coh}(S_{[i,j]}) & \text{en otro caso.} \end{cases}$$

Junto con $dp[j]$ se almacena $split[j] = \arg \max i$, que permite reconstruir la segmentación óptima recorriendo los cortes hacia atrás desde $j = n$.

4.2. Pseudocódigo

```
dp[0] = 0; dp[1..n] = -inf
for i in 0..n-1:
    for j in i+Lmin .. min(i+Lmax, n):
        if j - i == 1: coh = 1.0 # trivial, sin LLM
        else: coh = LLM(S[i:j]) # seccion 5
        gain = coh * (j-i) - (lambda if j-i==1 else 0)
        if dp[i] + gain > dp[j]:
            dp[j] = dp[i] + gain; split[j] = i
# reconstruccion desde split[n] hacia atras
```

4.3. Complejidad

Hay $O(n \cdot L_{\text{máx}})$ segmentos candidatos; cada uno requiere una llamada al LLM (la operación dominante) y trabajo $O(1)$ adicional, más la reconstrucción en $O(n)$. La complejidad total es

$$O(n \cdot L_{\text{máx}}) \text{ llamadas al LLM.}$$

Para $n = 12$ y $L_{\text{máx}} = 6$ son 57 candidatos por documento, de los cuales 12 son singletons resueltos sin LLM (45 llamadas reales). El parámetro $L_{\text{máx}}$ es así el control directo del coste de inferencia. El algoritmo es *exacto*: garantiza el óptimo global de J dentro de $\Sigma_{L_{\text{mín}}, L_{\text{máx}}}$, a diferencia de las heurísticas voraces de corte local.

5. Rol del LLM en el sistema

El LLM implementa exclusivamente la función de coherencia $\text{coh}(\cdot)$: es el *evaluador semántico* del sistema, mientras que la optimización corre por cuenta de la programación dinámica. Esta separación permite cambiar de modelo sin tocar el algoritmo.

5.1. Configuración

- **Modelo:** Qwen 2.5-3B-Instruct, cuantización Q4_K_M (archivo GGUF de ~ 2 GB), ejecutado localmente con llama-cpp-python 0.3.29.
- **Parámetros de inferencia:** contexto de 2048 tokens, 4 hilos de CPU, `temperature = 0` (evaluación determinista), `max_tokens = 1`, `logprobs` activado con `top_logprobs = 10`.

5.2. Puntuación por probabilidad de respuesta binaria

A cada segmento candidato se le formula, usando la plantilla de chat del modelo, la pregunta binaria:

```
[system] Eres un evaluador de cohesión temática. Respondes únicamente Sí o No.
[user] ¿Tratan todas las oraciones del siguiente segmento un mismo tema central? Responde únicamente Sí o No.

Segmento: <texto del segmento>
```

y se lee la distribución de probabilidad del *primer token* de la respuesta. La coherencia se define como la probabilidad normalizada de la respuesta afirmativa:

$$\text{coh}(S) = \frac{P(\text{¡Sí!})}{P(\text{¡Sí!}) + P(\text{¡No!})}.$$

Esto convierte una pregunta binaria en una puntuación continua en $[0, 1]$ y calibrada: segmentos claramente monotemáticos obtienen valores $\approx 1,0$, mezclas claras $\approx 0,0$, y los casos dudosos valores intermedios (p. ej. 0,12 para un par de oraciones que continúan un tema sin nombrarlo explícitamente).

5.3. Iteración de diseño: por qué no pedir un número directamente

La primera versión del evaluador pedía al modelo un número decimal entre 0 y 1. Durante la experimentación se detectaron y corrigieron tres fallos sucesivos, que se documentan por su valor metodológico:

1. **Truncamiento por tokens de parada:** con `stop = ["\n", " ", ",", ""]`, el primer token generado era un espacio y la generación se detenía al instante, devolviendo cadena vacía; el sistema caía silenciosamente en el valor por defecto 0,5 para *todos* los segmentos, y la DP optimizaba sobre una señal constante (segmentación resultante arbitraria).
2. **Formato de completado plano:** consultar el modelo *instruct* sin su plantilla de chat producía respuestas degeneradas (0,0 para cualquier segmento, coherente o no).
3. **Escalas numéricas mal calibradas:** con la plantilla de chat, pedir un decimal en $[0, 1]$ produjo salidas binarias (1,0 o 0,0) inconsistentes entre segmentos equivalentes, y pedir un entero de 0 a 10 colapsó casi todo a 5: un modelo de 3B parámetros no calibra bien escalas numéricas finas.

La lección es general: para modelos pequeños, una pregunta binaria evaluada por *logprobs* es más fiable que pedir puntuaciones numéricas, porque explota la capacidad del modelo de responder Sí/No (tarea en la que sí es competente) y obtiene la granularidad continua de la distribución de probabilidad, no del texto generado.

6. Metodología experimental

6.1. Métricas

Sea n el número de oraciones, R el conjunto de fronteras de referencia y H el de fronteras predichas (una frontera se identifica con el índice de la última oración de un segmento).

- P_k : se desliza una ventana de k oraciones ($k =$ mitad de la longitud media de segmento de referencia) y se mide la proporción de posiciones en las que referencia e hipótesis discrepan sobre si los dos extremos de la ventana caen en el mismo segmento. Penaliza fronteras desplazadas de forma proporcional a la distancia. 0 = perfecto.
- **WindowDiff**: variante de P_k que compara el *número* de fronteras dentro de cada ventana, corrigiendo la tendencia de P_k a infra-penalizar falsos positivos. 0 = perfecto.
- **Precisión, exhaustividad y F_1 de fronteras**: coincidencia exacta entre H y R ; es la métrica más estricta (no da crédito parcial a fronteras desplazadas una oración).

6.2. Configuraciones comparadas

Configuración	$L_{\text{máx}}$	$L_{\text{mín}}$	Descripción
Baseline uniforme	—	—	divide en k bloques iguales, $k =$ núm. segm. ref. (sin LLM)
DP $L_{\text{máx}} = 3$	3	1	ventana corta: no puede formar segmentos de 4
DP $L_{\text{máx}} = 6$	6	1	configuración estándar
DP $L_{\text{máx}} = 8$	8	1	ventana larga: más llamadas al LLM
DP $L_{\text{mín}} = 2$	6	2	prohíbe singletons

Cuadro 2: Configuraciones del experimento. El baseline uniforme recibe el número verdadero de segmentos k , información que las variantes DP *no* conocen, por lo que es un baseline exigente.

6.3. Protocolo y entorno

Cada configuración se evalúa sobre los 6 documentos y se promedian las métricas. La inferencia es determinista (`temperature = 0`), por lo que los resultados son reproducibles

con una sola corrida (`python evaluate.py -compare`). Entorno: Apple M5 (CPU, 4 hilos), Python 3.9, llama-cpp-python 0.3.29; $\sim 0,5$ s por llamada al LLM. Los registros completos de ejecución se conservan en `evidencias/` y los resultados numéricos en `resultados.json` y `resultados_comparacion.json`.

7. Resultados y análisis

7.1. Comparación entre configuraciones

La tabla 3 resume las métricas promediadas sobre los 6 documentos para cada configuración (los resultados completos, incluido el detalle por documento, están en `resultados_comparacion.json`).

Configuración	$L_{\text{máx}}$	$L_{\text{mín}}$	$P_k \downarrow$	WD $\downarrow$	$F_1 \uparrow$	Llamadas LLM
Baseline uniforme	–	–	0,0741	0,0556	0,9000	0
DP $L_{\text{máx}} = 3$	3	1	0,4741	0,5426	0,5172	114
DP $L_{\text{máx}} = 6$	6	1	0,0741	0,0370	0,9444	240
DP $L_{\text{máx}} = 8$	8	1	0,0741	0,0370	0,9444	294
DP $L_{\text{mín}} = 2$	6	2	0,0000	0,0000	1,0000	210

Cuadro 3: Métricas promedio por configuración (6 documentos). Las llamadas al LLM son el total sobre el dataset; los segmentos de una oración se resuelven sin LLM.

7.2. Resultados por documento

La tabla 4 desglosa por documento las dos configuraciones más relevantes: la estándar ($L_{\text{máx}} = 6$, $L_{\text{mín}} = 1$) y la ganadora ($L_{\text{máx}} = 6$, $L_{\text{mín}} = 2$).

Doc	Ref.	Tamaños	DP $L_{\text{mín}} = 1$			DP $L_{\text{mín}} = 2$		
			P_k	WD	F_1	P_k	WD	F_1
D01	3	4+4+4	0	0	1	0	0	1
D02	3	4+4+4	0	0	1	0	0	1
D03	2	4+4	0	0	1	0	0	1
D04	3	4+4+4	0	0	1	0	0	1
D05	3	4+4+4	0	0	1	0	0	1
D06	3	3+5+2	0,4444	0,2222	0,6667	0	0	1

Cuadro 4: Resultados por documento. Ambas configuraciones son perfectas en los documentos de segmentos uniformes; solo la variante con mínimo de segmento igual a 2 resuelve también el documento de segmentos no uniformes.

En D06, la configuración ganadora recupera exactamente las fronteras de referencia $\{3, 8\}$ (tras las oraciones 3 y 8), identificando los tres temas (cine, océanos, ajedrez) pese a sus tamaños dispares de 3, 5 y 2 oraciones. La configuración estándar, en cambio, predice 5 segmentos: acierta el bloque de cine y el inicio de océanos, pero fragmenta las últimas oraciones en singletons.

7.3. Análisis

La ventana debe cubrir el tamaño real de los temas. Con $L_{\text{máx}} = 3$, menor que el tamaño típico de segmento (4–5 oraciones), el algoritmo está estructuralmente obligado a partir cada tema en dos, y las métricas se desploman ($P_k = 0,47$, próximo al azar). Con $L_{\text{máx}} \in \{6, 8\}$ los resultados son idénticos entre sí: una vez que la ventana cubre el segmento más largo del

dataset (5 oraciones), ampliarla solo añade coste ($240 \rightarrow 294$ llamadas, $+22\%$) sin ningún beneficio. En la práctica conviene el menor $L_{\text{máx}}$ que cubra el tamaño de tema esperado.

La restricción estructural supera a la penalización suave. El único fallo de la configuración estándar (D06) es una partición degenerada: como las oraciones sueltas tienen coherencia trivial 1,0 y $\lambda = 0,05$ es pequeña, aislar las oraciones finales resulta rentable cuando el LLM asigna coherencia < 1 a un segmento legítimo pero breve (el bloque de ajedrez, de 2 oraciones, no repite el tema explícitamente). Prohibir directamente los singletons ($L_{\text{mín}} = 2$) elimina esa vía de escape y produce segmentación perfecta en todo el dataset, sin coste adicional (210 llamadas, incluso menos que la estándar). Aumentar λ lograría un efecto similar, pero exigiría calibrar un hiperparámetro continuo; la restricción dura es más simple y robusta cuando se sabe que ningún tema real ocupa una sola oración.

El baseline uniforme solo funciona cuando el dataset lo favorece. El baseline divide en k bloques iguales *conociendo el número verdadero de segmentos*, información que las variantes DP no reciben. Aun así: en D01–D05 es perfecto porque los segmentos de referencia son exactamente uniformes (artefacto de construcción del dataset), pero en D06 cae a $F_1 = 0,4$, mientras que DP $L_{\text{mín}} = 2$ mantiene $F_1 = 1$. Es decir, el sistema con LLM extrae de la semántica del texto lo que el baseline solo consigue cuando la estructura del documento coincide con su suposición y además se le regala k .

Coste computacional. La corrida completa de comparación (5 configuraciones, 858 llamadas al LLM) tomó 5 minutos en CPU ($\approx 0,35$ s por llamada), y la evaluación estándar del dataset completo, 86 segundos. El coste escala linealmente con $n \cdot L_{\text{máx}}$, conforme al análisis de la sección 4.

Sobre la señal del LLM. Inspeccionando los registros (*evidencias/*), la probabilidad $P(\text{ñSiz})$ se comporta como se esperaba: los 17 segmentos de referencia obtienen coherencia $\approx 1,0$, los candidatos que cruzan fronteras temáticas $\approx 0,0$, y los sub-segmentos que continúan un tema sin nombrarlo reciben valores intermedios. Esta separación nítida explica que la DP encuentre el óptimo correcto: la señal del evaluador es prácticamente un oráculo sobre este dataset, y las diferencias entre configuraciones provienen de las restricciones estructurales ($L_{\text{mín}}$, $L_{\text{máx}}$), no de errores del LLM.

8. Limitaciones y posibles mejoras

8.1. Limitaciones

- **Dataset pequeño y de fronteras nítidas:** 6 documentos con temas deliberadamente distantes. No mide el comportamiento ante transiciones graduales de tema, textos ruidosos o dominios especializados, y las métricas promedio tienen alta varianza con tan pocos documentos.
- **Coste de inferencia:** $O(n \cdot L_{\text{máx}})$ llamadas al LLM hace que documentos largos sean costosos ($n = 100$, $L_{\text{máx}} = 8 \Rightarrow \sim 800$ llamadas, unos 7 minutos en CPU).
- **$L_{\text{máx}}$ acota el tamaño real de los temas:** si un tema ocupa más de $L_{\text{máx}}$ oraciones, el sistema se ve forzado a partirlo.
- **La coherencia se evalúa por segmento aislado:** la función objetivo no compara segmentos vecinos, de modo que no penaliza explícitamente cortar un tema en dos mitades coherentes (solo la ponderación por longitud y λ lo desincentivan).

- **Sensibilidad al evaluador:** como mostró la iteración de diseño (sección 5), la calidad del sistema completo depende críticamente de que la señal del LLM esté bien calibrada; un cambio de modelo exige revalidar el prompt.
- **Segmentación por oraciones:** los cortes solo pueden caer en fronteras de oración detectadas por NLTK; errores del tokenizador se propagan.

8.2. Posibles mejoras

- **Caché y paralelización de llamadas:** memorizar $\text{coh}(S_{[i,j]})$ (segmentos repetidos entre configuraciones) y evaluar candidatos en lote aprovecharía mejor la GPU/CPU.
- **Señal de disimilitud entre vecinos:** añadir al objetivo un término que premie la *baja* coherencia entre el final de un segmento y el inicio del siguiente, penalizando cortes dentro de un mismo tema.
- **Preselección barata de candidatos:** usar embeddings (p. ej. similitud coseno entre oraciones consecutivas) para descartar candidatos obvios y reservar el LLM para los casos dudosos, reduciendo las llamadas en un orden de magnitud.
- **Datasets estándar y textos reales:** evaluar sobre corpus públicos de segmentación (Choi, Wiki-727k) y sobre documentos con transiciones suaves, con más documentos por configuración.
- **Modelos mayores o especializados:** comparar la señal de coherencia de modelos 7B–14B o de un modelo de embeddings fino, midiendo el compromiso calidad/coste.
- **Número de segmentos como restricción:** permitir fijar m (segmentos deseados) mediante una DP bidimensional $dp[j][m]$, útil cuando el usuario conoce la estructura esperada.